

Document 08 — RAG (Retrieval-Augmented Generation)

This is for self-study and live mentoring. Full plan: [CURRICULUM.md](#)

Prerequisites

[07_ai_agents](#) (Project 2 done)

How to Read & Practice This Document

- **What:** grounding a model's answers in your own real documents.
- **Why:** this is the most common real-world pattern for "answer questions about my data," and also the most common place where cost and quality trade-offs get made badly.
- **When:** when the model needs facts it wasn't trained on, or facts that change too often to write directly into a prompt.
- **How to practice:** 1. Read the material once, just to get the shape of it. 2. Do the **Basic** similarity exercise closed-book, then check it. 3. Do the **Intermediate/Real-world/Edge case** exercises with notes open — comparing chunking methods especially rewards actually trying it, not just reading about it. 4. Build the retriever without looking at an old solution — chunk something badly on purpose first, so you see the failure before you fix it. 5. Use **Hint 1** → **Hint 4** only after really trying. Ask for the full solution only if you say "**Show me the solution.**" 6. Before moving on, explain out loud the difference between a retrieval failure and a generation failure, with a real example of each from your own testing. If you can't, you're not done.

Jump to: [Core Concepts](#) · [Practice Exercises](#) · [Build Task](#)

The Story — what this document is actually building

Picture this: your Doc07 agent is genuinely good at using tools — but it still only knows what its training data knew, plus whatever's in the current conversation. Ask it a question about your own documents (your company's policies, your own notes, a PDF you just wrote) and it either says "I don't know" or, worse, confidently makes something up. RAG is how you fix that: you hand the model the actual relevant text, right in the prompt, at the moment it needs it.

That's four separate-sounding topics, but they're really one pipeline, in order. First, **chunking** splits your documents into small pieces, because searching one giant document is slow and imprecise — you want the one paragraph with the answer, not the whole file. Second, **embeddings** turn each chunk (and later, each question) into a list of numbers, built so that pieces of text with similar meaning end up close together in that number space. Third, a **vector store** holds all those chunk-numbers and, given a question's numbers, quickly finds the closest matches — this is the actual "search" in RAG. Fourth, those matching chunks get stuffed into the prompt, and the model generates an answer grounded in real text instead of its own memory.

Here's the part that trips people up: every one of those four steps can go wrong independently, and a wrong final answer doesn't tell you which one broke. Cut a chunk in the wrong place, and the fact you needed gets split in half before it's even embedded. Retrieve too few chunks, and you miss the answer. Retrieve too many, and the model gets confused by irrelevant text ("lost in the middle"). Or every step works perfectly, and the model still misreads a chunk it was correctly handed. Learning to tell these apart — a retrieval failure vs. a generation failure — is the actual skill this document is teaching.

The **Build Task** ties all of this into one function: `retrieve(query) -> List[Document]`. It's not its own app — it becomes a *tool* that Project 3's agent calls whenever it needs to look something up in your documents, the same way a search-the-web tool would.

Core Concepts (read this first — everything you need is here)

Embeddings: meaning turned into geometry

An embedding model turns text into a list of numbers (a vector), built so that pieces of text with a *similar meaning* end up close together in that number space — "close" measured by a distance formula like cosine similarity. **Why this is useful:** it turns "find text with a similar meaning to this question" from a fuzzy language problem into plain math (comparing vectors), which a database can do fast, even across millions of documents. **How it works, worth actually knowing:** the embedding model is a separate model from the chat model, trained specifically so that text with related meaning ends up near each other in this number space — "the cat sat on the mat" and "a feline rested on the rug" are very different sentences, but end up as nearby vectors, while "the cat sat on the mat" and "stock prices rose today" end up far apart.

Chunking: why where you split matters more than it seems like it should

Before embedding, documents get split into smaller pieces ("chunks"), because embedding and searching whole documents is both slower and less precise — you want to find the exact paragraph that answers a question, not hand back an entire 50-page document. **Why the exact split point matters so much:** if a split happens right in the middle of the one sentence that has the actual answer, the search might return a chunk with only half the fact — technically "related," but useless. And this problem is invisible until you specifically test that one question. **When each method is**

worth it: splitting by a fixed number of characters is simplest, but doesn't know anything about structure (it can cut a sentence in half). Splitting by paragraph respects natural breaks, but gives uneven chunk sizes. Splitting by meaning (finding where the topic actually shifts) is the most accurate, but the most expensive to compute. There's no single right answer — it's a trade-off you make on purpose, for each type of document, which is exactly why the build task has you compare methods instead of just picking one blindly.

Vector stores and searching for the top matches

A vector store keeps all your chunk vectors organized so that, given a question's vector, it can quickly find the `k` closest ones (called top-k search) without comparing against every single chunk one by one. **Why `k` is a real design decision, not just a number you set once and forget:** too small (`k=1`) risks missing a useful chunk that scored just below the cutoff. Too large (`k=10` +) risks the "lost in the middle" problem below, and adds cost and delay for very little benefit. **How a question flows through the system:** the question text gets turned into a vector, using the *same* embedding model used for the documents (using different embedding models for the question and the documents gives meaningless distances) → compared against the stored chunk vectors → the closest `k` come back, usually with a score you can use as a cutoff to filter out weak matches, instead of always returning *something* no matter how unrelated.

"Lost in the middle": more text isn't automatically better

Studies on long-context models keep finding that models are more reliable using information near the *start* or *end* of a long prompt than information buried in the middle — so just stuffing in every retrieved chunk "to be safe" can actually make answers *worse*, even though the right information is technically in there somewhere. **Why this matters for how you build RAG:** finding relevant text isn't the whole job — it's finding the *right amount* of the *most* relevant text. More retrieved context isn't automatically better, which is exactly why choosing the right `k` (above) is a real design decision.

RAG changes what gets made up — it doesn't remove the problem

Grounding a model in real, retrieved text cuts down one specific failure (inventing facts out of nowhere), but brings in others: the model can still misread a chunk, mix up two chunks, or answer confidently from a chunk that's on-topic but doesn't actually contain the answer. **Why this distinction is the whole practical skill of debugging RAG:** when an answer is wrong, the fix depends completely on *where* it went wrong — bad retrieval (the right chunk never got found — fix chunking, embeddings, or `k`) vs. bad generation (the right chunk was found, but the model used it wrong — fix the prompt or the instructions). Treating every wrong RAG answer as "the model made it up" without checking which stage actually broke wastes time fixing the wrong layer.

Hybrid search: combining keyword and vector search

Pure embedding search is great at "similar meaning," but it's surprisingly bad at exact matches — a product SKU like `SKU-4821`, an error code like `ERR_TIMEOUT_502`, or a person's exact name can end up with *lower* similarity than you'd expect, because the embedding model is busy encoding overall meaning, not exact characters. A plain keyword search (the classic kind, matching literal words — often scored with an algorithm called BM25) catches exact matches like that instantly, but misses reworded questions that mean the same thing in different words. **Why real systems combine both:** hybrid search runs a keyword search and a vector search side by side, then merges the two ranked lists into one (a common way is Reciprocal Rank Fusion — a chunk that ranks well in *either* list gets a good combined score). **When to use it:** almost any real production RAG system with a mix of exact-match content (codes, IDs, names) and natural-language content benefits from hybrid search over embeddings alone. **How it works, in outline:**

```
def hybrid_search(query, k=5):
    keyword_hits = bm25_search(query, k=k)      # exact-word matching
    vector_hits = vector_search(query, k=k)      # meaning matching
    return reciprocal_rank_fusion(keyword_hits, vector_hits, k=k)
```

Most vector databases (Chroma, Weaviate, Qdrant) either support this natively or make it easy to bolt a keyword index on next to the vector one.

Reranking: a second, more careful pass over the top results

The first-pass vector search is built for speed — it has to scan potentially millions of chunks in milliseconds, so it uses a cheap, approximate comparison (comparing two vectors is fast, but the vectors themselves are a lossy summary of the text). A **reranker** is a smaller, slower, more accurate model whose only job is: given a question and a short list of candidate chunks, look at each one *properly* (reading the actual question and the actual chunk together, not just comparing pre-computed vectors) and re-score them. **Why this two-stage, "fast-then-precise" approach is common:** running the precise, expensive method over every chunk in a large knowledge base would be far too slow; running only the cheap, approximate method risks a mediocre final order. Doing both — cheap search narrows millions of chunks down to, say, the top 20, then the reranker carefully re-orders just those 20 — gets you most of the speed of the first method and most of the accuracy of the second.

```
candidates = vector_search(query, k=20)  # fast, approximate
reranked = reranker.rerank(query, candidates, top_n=5)  # slower, precise, on far
```

When to use it: anywhere search quality genuinely matters and you can afford one extra model call per query — it's a very common addition once a basic RAG pipeline is working but "close enough" isn't good enough anymore.

Metadata filtering

Every chunk can carry metadata alongside its text and vector — things like `source_document`, `date`, `user_id`, or `tags`. **Metadata filtering** narrows the search to only chunks matching some condition, either before the similarity search runs (searching only within a smaller subset) or as a filter applied to the results. **Why this matters for relevance:** if you know the user is asking about "the 2024 policy," filtering to `date == "2024"` before searching avoids ever comparing against older, irrelevant chunks, whatever their vector similarity happens to be. **Why this matters even more for access control:** if different users uploaded different documents, a search without a filter can return chunks from documents that user genuinely isn't allowed to see — a real security bug, not just a relevance annoyance. The filter has to be applied every time, as part of the query itself, not as an afterthought checked later.

```
retrieve(query="refund policy", filters={"user_id": current_user.id, "tag": "poli
```

When to use it: any system with more than one "owner" of the documents (multi-user apps, multi-tenant systems), or any system where documents are naturally grouped and a question is usually about just one group.

Evaluating a RAG system

"Does the answer look right" isn't a real evaluation — it doesn't scale past a handful of manual checks, and it doesn't tell you *which* stage of the pipeline to fix when something's wrong. There are two genuinely separate kinds of metrics here, and each can fail on its own. **Retrieval metrics** ask: did the right chunk even get found? (Common ones: did the correct chunk appear anywhere in the top-k results, and how high did it rank.) **Generation metrics** ask a completely different question: given that the model was handed the right chunk, did it actually use it correctly in the answer? A system can score perfectly on retrieval and still generate a wrong answer (a generation failure), or generate a perfectly reasonable-sounding answer built from the wrong chunk because retrieval never found the right one (a retrieval failure) — this is exactly the retrieval-vs-generation distinction from the Core Concepts topic above, just turned into something you can measure instead of eyeball. **When to use it:** this is only previewed here to connect it to the failure types you're already learning to spot by hand — [13_testing_evaluation_observability](#) covers building an actual evaluation suite (test sets, scoring, and LLM-as-a-judge) properly.

RAG-specific prompt injection: your own knowledge base can attack your own agent

[06_tools_function_calling](#) Core Concepts — "Prompt injection: when the attack comes from content, not the user" introduces prompt injection in general — this is its sharpest, most concrete version, because here the attacker's content isn't even flagged as "external." **The problem:** if *any* document that gets indexed into your vector store — an uploaded file, a scraped webpage, a user-submitted support ticket — can contain hidden instructions, those instructions get chunked and embedded right alongside your legitimate content. Nothing in the pipeline marks it as suspicious. When `retrieve()` later returns that chunk as one of the "top matches," the model receives it as *your own trusted knowledge base speaking* — which is actually more dangerous than an obviously-external

source like a fetched web page, because there's no reason built into the system to be suspicious of it at all.

Why this is easy to miss: a team can spend real effort locking down "don't let the agent fetch untrusted URLs" and still get hit, because the attack came in through document upload or ticket ingestion weeks earlier, sitting quietly in the vector store until some later query happens to retrieve it.

How to actually reduce it: sanitizing or scanning documents before they're chunked and embedded — stripping hidden text, flagging suspicious phrasing — is one layer, but it's a filter, not a guarantee. The more reliable layer is applying [06_tools_function_calling's "Prompt injection" topic's](#) "treat retrieved content as data, not instructions" framing exactly where it matters most here: at the `retrieve()` boundary, right before its output ever reaches the model.

```
def build_context(chunks: list[str]) -> str:
    # wrap retrieved chunks as labeled data, never as raw prompt text
    joined = "\n--\n".join(chunks)
    return f"<retrieved_context>\n{joined}\n</retrieved_context>"
```

The system prompt then tells the model, explicitly, that anything inside `<retrieved_context>` is material to summarize or quote from — never a command to follow, no matter how it's phrased. That one boundary is what keeps "your own knowledge base" from silently becoming an attacker's delivery channel.

Exposing your retriever as an MCP tool

What this actually is: everything in this document builds a `retrieve(query, k)` function — a plain Python function that only the script it lives in can call. If a teammate, a different app, or even Claude Desktop wants to search your documents, they can't — the function is trapped inside your one program. Turning it into an MCP Tool means wrapping that same function so it's reachable over a standard protocol, by any MCP-compatible client, not just your own code.

Why this matters, specifically: without MCP, "let someone else use my retriever" means writing a custom integration for every single client that wants it — a REST endpoint for one teammate's app, a different wrapper for an IDE plugin, something else again for a chatbot. Each one is separate work, and each one has to be maintained separately as your retriever changes. [06_tools_function_calling](#) covers MCP (Model Context Protocol) precisely because it removes this duplication: you expose your retriever *once*, as one MCP server, and every client — Claude Desktop, an IDE, an agent from Project 8, a teammate's own agent — connects to that same server the same standard way. You maintain one integration instead of N of them.

How it actually works, underneath: the wrapping itself is thin — you are not rewriting `retrieve()`, you're placing a small, standard-shaped function around it. `MCPServer` is the official SDK's helper class that turns a plain Python function into something MCP clients can discover and call: the `@mcp.tool()` decorator registers the function under a name (`search_documents`) and a

description (its docstring), which is exactly what a connecting client sees when it calls `list_tools()` — the same discovery mechanism covered in Doc06. The function body does nothing new; it just calls the `retrieve()` you already built and hands back the result, reshaped into the plain dict/list format MCP expects on the wire:

```
from mcp.server.mcpserver import MCPServer
from retriever import retrieve

mcp = MCPServer("knowledge-base")

@mcp.tool()
def search_documents(query: str, k: int = 3) -> list[dict]:
    """Search the knowledge base and return the top k matching chunks."""
    return retrieve(query, k)
```

Once this server is running, any MCP client — including one you never wrote and don't control — can call `search_documents("refund policy")` and get real, grounded results back, without knowing or caring whether your retriever uses Chroma, FAISS, or something else entirely underneath. That's the actual point of the protocol: the client only needs to know the tool's name and its expected inputs/outputs, never your implementation.

When to bother doing this: not for a one-off script only you run — that's needless overhead. Do it once your retriever is genuinely useful *outside* the one script you built it in: you also want Claude Desktop to search the same documents, or a teammate's separate agent needs the same knowledge base, or you're building Project 8's agent and want it to pull from a real, running server instead of importing your retriever code directly. Project 7 builds this exact pattern properly, end to end, with a real local database and a real running server — not just one wrapped function.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- [OpenAI — Embeddings guide](#) — what an embedding is, and how to make one.
- [LangChain — RAG concept](#) — chunking, retrieval, and generation as three separate jobs.
- [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks \(arXiv, Lewis et al.\)](#) — the original RAG paper.
- [Chroma docs](#) — the vector store you'll use locally (or [FAISS](#)).

Practice Exercises

Setup for this document's practice code: work inside `08_rag/` (same venv as before — if it's not active, `source .venv/bin/activate`). New package for this document: `pip install langchain langchain-openai chromadb` .

How to run each exercise: group your practice code by topic, not by difficulty level. If two exercises below are really about the same thing, save them together in ONE script named after that topic — for example, if two exercises are both about `.env` config, save both in one file like `env_config_practice.py` , with each level's version as its own clearly labeled section inside it. Run each topic's file directly, for example: `python chunking_practice.py` .

For this document, save your practice code as: - **Basic** (see embeddings as geometry, not theory) is its own topic — save it as `embedding_similarity_practice.py` . - **Intermediate** (chunking method changes the answer), **Edge cases** (when the answer spans two chunks), and **Failure** (a bad split, and a `k` comparison) are all about how you split documents into chunks and what that does to search — save them together as `chunking_practice.py` , one section per level. - **Real-world** (build and search a real knowledge base) is its own topic — save it as `knowledge_base_search_practice.py` .

Why group by topic instead of by level: if you save each exercise by difficulty level instead, the different versions of the same idea end up scattered across separate files, and you can never see how one topic grows from simple to harder in one place. Grouping by topic keeps that growth visible — open one file, and you see the whole journey for that one thing, from basic to advanced, side by side.

Jump to an exercise: [Basic](#) · [Intermediate](#) · [Real-world](#) · [Edge cases](#) · [Failure](#) · [Build Task](#)

Basic — see embeddings as geometry, not theory

- **What:** embed 5 short sentences and work out cosine similarity between every pair, confirming "similar meaning = close vectors."
- **Why:** this is the one fact the entire rest of RAG rests on — you need to see it work with real numbers, not just accept it as a claim.
- **When you'll hit this for real:** debugging any search result that looks wrong — you'll come back to "are these actually close in vector space" as your first check.
- **How to code it:** get embeddings for 5 sentences (2 similar in meaning, 3 unrelated) via the OpenAI embeddings API, compute cosine similarity for every pair with a small function, and confirm the 2 similar ones score highest against each other.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Intermediate — chunking method changes the answer

- **What:** split the same document 3 different ways (by character count, by paragraph, by meaning) and compare search quality for the same question.

- **Why:** this document's Core Concepts claim chunking is a real design decision, not a detail — this exercise is where you prove that to yourself with a real before/after.
- **When you'll hit this for real:** the Build Task below, and any real knowledge base where naive fixed-size chunking quietly breaks an answer.
- **How to code it:** write `chunk_by_chars()` and `chunk_by_paragraph()`, run the same document through both, embed and store both sets, then search both with the same question and compare which chunk actually surfaces.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Real-world — build and search a real knowledge base

- **What:** write 5-10 short real documents yourself, and search against them.
- **Why:** made-up test documents you actually wrote are the fastest way to know, immediately, whether a search result is right or wrong — you already know the answer.
- **When you'll hit this for real:** this document's own Build Task, which becomes Project 3's search tool.
- **How to code it:** write 5-10 short `.txt` / `.md` files on one topic you know well, embed and store them with Chroma, then run 3 questions against them and check the results by eye.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Edge cases — when the answer spans two chunks

- **What:** a question whose answer spans two chunks next to each other — check whether search returns both.
- **Why:** this is one of the most common real RAG failures, and it's invisible until you specifically construct a test case for it.
- **When you'll hit this for real:** any document where a key fact is split across a chunk boundary by chance — you won't know it's happening until an answer comes back half-right.
- **How to code it:** deliberately write one document where the answer to your test question spans across a chunk boundary you control, search for it, and check whether both chunks come back in your top-k.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Failure — a bad split, and a `k` comparison

- **What:** split a document badly on purpose (cutting mid-sentence, mid-fact) and show the resulting search-then-answer failure. Then compare search quality and speed for `k=1`, `k=3`, and `k=10`.
- **Why:** you need to have caused this failure once, on purpose, so you recognize the *shape* of it immediately when it happens by accident later.
- **When you'll hit this for real:** any time you reach for the simplest chunking method (fixed character count) without checking where the cuts actually land.

- **How to code it:** force a chunk split in the middle of a sentence containing your test answer, search for it, and watch the result come back incomplete. Then run the same query at `k=1` , `k=3` , `k=10` and note both the results and how long each takes.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Build Task — Retriever Module

Stuck on the Build Task? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Goal: a `retrieve(query) -> list[Document]` function — this becomes a *tool* in Project 3, not its own separate app.

Requirements:

- Takes in a small set of local documents, splits them, embeds the pieces, and stores them in a vector store (Chroma or FAISS).
- `retrieve(query: str, k: int) -> list[Document]` returns the top-k closest matches, with which document and which chunk each came from.
- Handles an empty (or nearly empty) set of documents without crashing (returns an empty list, not an error).

Inputs: a folder of local text documents (write 5-10 short ones on a topic you pick), and a question at search time.

Outputs: a ranked list of chunks, with scores and where each one came from.

Constraints: the chunking method must be its own named, swappable piece (you'll compare methods) — not hardcoded inline.

Suggested files:

```
08_rag/
├── ingest.py
├── chunking.py
├── retriever.py
├── docs/                (your sample documents, plain text/markdown)
└── test_retriever.py
```

Functions/Components to build:

- `chunking.py` → at least two chunking methods, like `chunk_by_chars()` , `chunk_by_paragraph()`
- `ingest.py` → `build_vector_store(doc_folder: str) -> VectorStore`
- `retriever.py` → `retrieve(query: str, k: int = 3) -> list[Document]`

Expected Behavior

- A question that closely matches a document should return that document's chunk(s) near the top.
- An off-topic question should return low scores (or nothing, with a cutoff set) — not confidently-wrong top results.
- Different chunking methods should measurably change results for at least one question in your test set.

Test Cases

Scenario	Expected
Question matches a document word-for-word	That chunk ranks #1
Question is reworded, not exact	The right chunk still ranks highly
Question is off-topic vs. the whole set of documents	Low scores across the board, or empty results with a cutoff set
Empty set of documents	<code>retrieve()</code> returns <code>[]</code> , no error

Break-It / Debug Preview

- A chunk split that cuts the one fact a question needs in half.
- A search that confidently returns the wrong documents, because of an embedding/question mismatch.
- Full debugging drill in [14_debugging_lab](#).

Interview Topics Preview

- Chunking trade-offs · picking an embedding model · why RAG doesn't remove the "making things up" problem · telling a retrieval failure apart from a generation failure.

Move On When

You can tell whether a bad RAG answer is a retrieval problem or a generation problem — that one skill is the whole point of this document. Full details: [CURRICULUM.md §4](#).

Stuck? Ask for **Hint 1** or **Hint 2** (each has Basic/Intermediate/Advanced depth). Ask for the full solution only if you say **"Show me the solution."**